

Virtual Threads in Debezium Engine

Vojtěch Juránek

Red Hat

November 18th 2025, Debezium F2F meeting, Prague

Java threads

- 1:1 mapping to OS threads.
- Memory heavy (MBs).
- Context switching is time consuming.
- Creation is expensive.

Thread pooling

- Doesn't solve memory overhead and context switching, just creating overhead.
- Issues with `ThreadLocal` and thread interrupts.
- Mitigate context switching → core-per-thread architecture.

Asynchronous programming

- Breaking tasks into smaller ones.
- Better CPU utilization, as blocking tasks (typically IO) are waiting in a queue instead blocking thread.
- Chaining usually via callbacks.

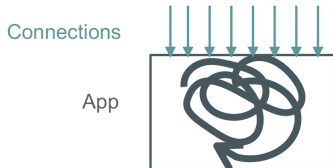
Synchronous vs. asynchronous

Synchronous:

- Simple to reason about it.
- Blocking.
- Less scalable.

Asynchronous:

- Better scales.
- Harder to reason about it.
- “Callback hell”.
- To get maximum benefit from it, all parts have to be asynchronous.
- Can be problematic to make it working with legacy synchronous code.



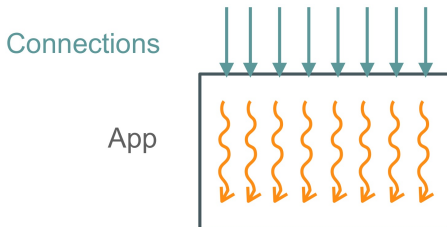
Source:

<http://cr.openjdk.java.net/~alanb/loom/Devoux201>

Java threads

What about having positives from both approaches:

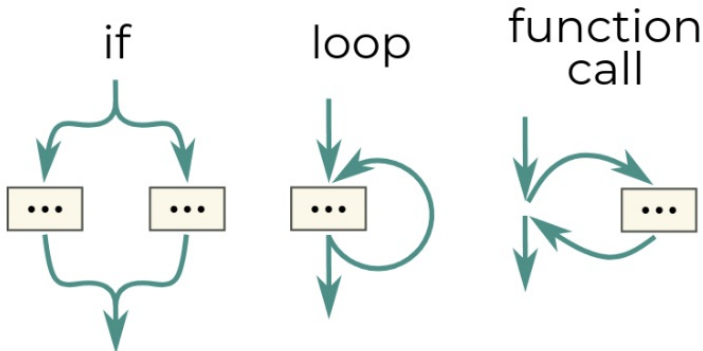
- Code like synchronous code.
- Work like asynchronous code.



Source: <http://cr.openjdk.java.net/~alanb/loom/Devoxx2018.pdf>

Structured programming

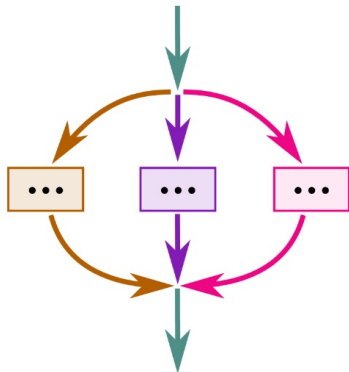
Replacing `goto` with `if-else`, `loop` statements and function calls in 60s:



Source: <https://vorpus.org/blog/notes-on-structured-concurrency-or-go-statement-considered-harmful/>

Structured concurrency

- Launching a task in a thread is like using `goto`.
- Structured concurrency is similar to using `if-else` and `loop` statements instead of `goto`.



Source: <https://vorpus.org/blog/notes-on-structured-concurrency-or-go-statement-considered-harmful/>

- Virtual threads (JEP 444) available and supported.
- Structured concurrency (JEP 505) is also available, but only as a preview feature (fifth preview).

First candidate for virtual threads - Asynchronous engine

```
1 final List<Future<R>> recordFutures = new ArrayList<>(  
    records.size());  
2 records.stream().forEachOrdered(r -> recordFutures.add(  
    recordService.submit(new ProcessingCallables.  
    TransformAndConvertRecord<R>(r, transformations,  
    convertor))));  
3  
4 final List<R> convertedRecords = new ArrayList<>(  
    recordFutures.size());  
5 for (Future<R> f : recordFutures) {  
6     R record = f.get(); // we need the whole batch,  
        eventually wait forever  
7     if (record != null) {  
8         convertedRecords.add(record);  
9     }  
0 }
```

Possible roadmap

- Start replacing execution in threads by virtual threads in Debezium engine - pretty isolated and self-contained component.
- Measure, if there is any performance gain using JMH
- If there are no issue apply virtual threads also in rest of the Debezium.
- Once structured concurrency is available, apply start to use it as well.

OR

- Wait until structured concurrency is supported and start with migration to virtual threads only after that.
- Wait for next LTS (Java 29 in September 2027)?